

to various other Collection classes. These methods are provided on top of the inherited methods from the `tuple` superclass.

- `_make()` : This method is a class method and is used to build a `namedtuple` by providing an iterable, which is passed as an argument to this method. The order of iterable should match the order of `keys` in the `namedtuple`.

```
>>> emp = Employee._make(['David', '10', 'Mumbai']);
>>> emp
Employee(name='David', age='10', address='Mumbai')
```

- `_asdict()` : It returns an `OrderedDict` instance by mapping each of the values against their respective keys.

```
>>> emp._asdict()
OrderedDict([('name', 'David'), ('age', '10'), ('address', 'Mumbai')])
```

- `_replace(**kwargs)` : The `_replace` method accepts `keyword arguments` as a parameter and returns a new instance by replacing those keys in the `namedtuple`, if supplied.

```
>>> emp._replace(name='Harry', age='18')
Employee(name='Harry', age='18', address='Mumbai')
```

Alternatively, if we wish to convert a dictionary into a `namedtuple`, we can directly pass the dictionary into the constructor using the `double-star-operator` (`**`)

```
>>> new_employee={'name':'John','age':'25','address':'New York'}
>>> Employee(**new_employee)
Employee(name='John', age='25', address='New York')
```

Often we might encounter use cases wherein we require all the fields of the `namedtuple`, that we had provided during initialization. `_fields` attribute, when called on a `namedtuple` class or the class object, we get the fields that were initialized.

```
>>> emp._fields #called on instance
('name', 'age', 'address')
>>> Employee._fields #called on class
('name', 'age', 'address')
```

We can also use `_fields` to merge two `namedtuples` and build a hybrid tuple.

```
>>> Manager = namedtuple('Manager', ('manager_name', 'manager_phone'))
>>> EmployeeManager = namedtuple(EmployeeManager,
```